

R-Code Beispiele aus WDDA Vorlesung 01

WDDA – Datenmanagement und Datenanalyse

Prof. Dr. Ulrich Matter

2026-04-29

Inhaltsverzeichnis

1	Einleitung	1
2	Import eines Datensatzes	1
3	R Syntax-Stile	2
4	Navigation in Dataframes	2
5	Verwendung von <code>attach()</code> und <code>detach()</code>	3
6	Verwendung von <code>length()</code>	3

1 Einleitung

Dieses Dokument fasst alle R-Code-Beispiele der WDDA Lecture 01 Folien zusammen. Es enthält zusätzliche Kommentare, Erklärungen und Beispiele, um das Verständnis für die einzelnen Code-Snippets zu erleichtern.

2 Import eines Datensatzes

In diesem Abschnitt importieren wir einen Datensatz mit dem **readxl**-Paket. Dabei erkennt `read_excel()` automatisch die Datentypen. Zur Überprüfung der importierten Datenstruktur kannst Du `str()` verwenden.

```
library(readxl) # Lade das Paket readxl zum Importieren von Excel-Dateien
markets <- read_excel("WDDA_01.xlsx", sheet = "Exchanges") # Importiere die Daten aus
  ↪ dem Blatt "Exchanges"
```

Zusätzlicher Tipp: Prüfe die Struktur des importierten Dataframes:

```
str(markets) # Zeigt die Struktur des Dataframes 'markets'
```

```
## tibble [21 x 3] (S3: tbl_df/tbl/data.frame)
## $ exchange: chr [1:21] "Athens" "Borsa Italiana" "Bratislava" "Bucharest" ...
## $ trades : num [1:21] 599192 5921099 111 79921 298871 ...
## $ turnover: num [1:21] 2009.8 44385.9 0.1 45.3 1089.6 ...
```

Hinweis: R speichert das Objekt `markets` als einen Dataframe.

3 R Syntax-Stile

R bietet verschiedene Möglichkeiten, Code zu schreiben. Die zwei Hauptstile sind:

1. **Klassische (verschachtelte) Syntax:** Funktionen werden von innen nach aussen gelesen
2. **Pipe-Syntax:** Funktionen werden von links nach rechts gelesen (wie in einer Pipeline)

Hier ein Vergleich der beiden Stile:

```
# Klassische Syntax (verschachtelt)
mean(sqrt(abs(x))) # Lese von innen nach aussen: erst abs(), dann sqrt(), dann mean()

# Pipe-Syntax mit |>
x |> abs() |> sqrt() |> mean() # Lese von links nach rechts: erst abs(), dann sqrt(),
  ↳ dann mean()

# Komplexeres Beispiel:
# Klassisch
subset(transform(data, z = x + y), z > 0)

# Mit Pipe
data |>
  transform(z = x + y) |>
  subset(z > 0)
```

Die Pipe-Syntax (`|>`) wurde in R 4.1.0 eingeführt und ist jetzt der empfohlene Stil für lesbaren Code. Der ältere Pipe-Operator (`%>%`) aus dem `magrittr`-Paket funktioniert ähnlich.

Tipp: In RStudio können Sie die Tastenkombination **Ctrl + Shift + M** verwenden, um den Pipe-Operator einzufügen. Um zwischen dem nativen (`|>`) und dem `magrittr` (`%>%`) Pipe-Operator zu wechseln, gehen Sie zu: Tools -> Global Options -> Code -> “Use native pipe operator” (Quelle: Stack Overflow)

4 Navigation in Dataframes

Hier zeigen wir verschiedene Methoden, um ein Dataframe zu inspizieren und zu unterteilen. Jede Methode ist für unterschiedliche Aufgaben nützlich.

```
# Zeige das gesamte Dataframe an
markets

# Zeige die Spaltennamen des Dataframes an
names(markets)

# Klassische Subsetting-Beispiele:
markets$trades      # Greife auf die Spalte 'trades' zu
markets[3,]         # Greife auf die dritte Zeile zu
markets[,2]         # Greife auf die zweite Spalte zu
markets[3,2]        # Greife auf das Element in der 3. Zeile und 2. Spalte zu

# Mit tidyverse-Funktionen für eine lesbarere Syntax:
markets |> pull(trades) # Extrahiere die Spalte 'trades' als Vektor
markets |> slice(3)     # Greife auf die dritte Zeile zu
markets |> select(2)    # Wähle die zweite Spalte anhand ihrer Position aus
```

```
markets |> select(trades)      # Wähle die Spalte 'trades' anhand ihres Namens aus
markets |> slice(2) |> select(trades)  # Wähle zuerst die zweite Zeile und anschliessend
  ↳ die Spalte 'trades'
```

Zusätzliches Beispiel: Zeige die ersten sechs Zeilen des Dataframes an:

```
head(markets)  # Zeige die ersten sechs Zeilen des Dataframes
```

```
## # A tibble: 6 x 3
##   exchange      trades turnover
##   <chr>         <dbl>     <dbl>
## 1 Athens          599192    2010.
## 2 Borsa Italiana 5921099   44386.
## 3 Bratislava       111         0.1
## 4 Bucharest       79921        45.3
## 5 Budapest       298871    1090.
## 6 Bulgarian      14040         64.4
```

5 Verwendung von attach() und detach()

Die Funktion `attach()` erleichtert den Zugriff auf die Variablen eines Dataframes, indem sie diese dem Suchpfad hinzufügt. Allerdings kann dies zu Namenskonflikten führen, wenn mehrere Objekte denselben Namen haben. Eine Alternative ist die Verwendung von `with()`, die den Gültigkeitsbereich einschränkt.

```
# Hänge das Dataframe an, sodass dessen Spalten direkt zugänglich sind
attach(markets)

# Jetzt kann auf 'trades' direkt zugegriffen werden, ohne den Dataframe-Namen zu
  ↳ verwenden
trades

# Entferne das Dataframe aus dem Suchpfad, um Namenskonflikte zu vermeiden
detach(markets)

# Nach dem Detach führt der Zugriff auf 'trades' ohne Dataframe-Namen zu einem Fehler:
trades
```

Zusätzlicher Hinweis: Eine Alternative zu `attach()` ist die Verwendung von `with()`, die nur innerhalb eines bestimmten Ausdrucks wirkt:

```
with(markets, summary(trades))  # Fasst die Spalte 'trades' zusammen, ohne das Dataframe
  ↳ dauerhaft anzuhängen
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      111   31167  433545 2710837 2799329 16539588
```

6 Verwendung von length()

Die Funktion `length()` verhält sich bei Dataframes und Vektoren unterschiedlich. Bei einem Dataframe gibt `length()` die Anzahl der Spalten (Variablen) zurück, nicht die Anzahl der Zeilen (Beobachtungen). Für die

Zeilenzählung solltest Du `nrow()` verwenden.

```
# Bei einem Dataframe gibt length() die Anzahl der Spalten zurück  
markets |> length()  
  
# Bei einem Vektor gibt length() die Anzahl der Elemente (Beobachtungen) zurück  
trades |> length()  
  
# Um die Anzahl der Zeilen (Beobachtungen) eines Dataframes zu erhalten, verwende nrow():  
nrow(markets)
```

Zusätzlicher Tipp: Mit `ncol()` kannst Du die Anzahl der Spalten ermitteln:

```
ncol(markets)
```

```
## [1] 3
```